

FixMate

On-Demand Service Booking — Single Page Application

CE3404 GUI Programming · Mini Project Report

REG NO: EG/2023/5466

Overview

FixMate is a fully-featured on-demand home services platform built as a single-page application for the CE3404 GUI Programming module. The project demonstrates the practical application of modern frontend technologies — Vue 3, TypeScript, Pinia, and Tailwind CSS — while simulating real-world scenarios like authentication, role-based access, live booking management, and push notifications.

The application serves two distinct user types: customers who need help around the house, and workers who offer their skills. Each role gets its own tailored experience, with thoughtful flows for everything from browsing services and booking a professional, to a worker accepting a job and marking it complete.

1. Features Implemented

Authentication & Session Management

The app uses the DummyJSON `/auth/login` API endpoint to simulate real authentication. A JWT token is returned and stored in `localStorage`, so sessions survive page refreshes without requiring the user to log in again. During registration, form validation runs client-side before any API call is made. Role selection — either Customer or Worker — happens at login, and route guards enforce that each role can only reach pages built for them.

Customer Experience

HOME PAGE

The landing page is designed to make a strong first impression. It includes an auto-sliding advertisement carousel, a grid of eight service categories, a curated list of top professionals presented as cards, a step-by-step 'How It Works' section, and a testimonials area featuring customer reviews. Four animated platform stat cards give visitors an at-a-glance sense of scale.

SERVICES & WORKERS

The Services page gives customers two ways to explore: a Services view and a Workers view, toggled with a tab switcher. Real-time search with debounce keeps the experience snappy, and category filter buttons let customers narrow down exactly what they need. Clicking a worker opens a dedicated profile page with ratings, years of experience, specializations, and a list of recently completed jobs.

BOOKING FLOW

Booking is a clean three-step process. First, the customer selects a worker. Second, they enter a date, time, and address. Third, they review an order summary and choose their payment method — either cash on delivery or card payment. The modal or bottom sheet keeps everything contextual without navigating away from the page.

POST-BOOKING

Customers can view their entire booking history with live status tracking. A notification centre surfaces real-time updates when a booking is accepted, rejected, or completed. Dark mode is available as a toggle and persists across sessions.

Worker Experience

DASHBOARD

Workers land on a dashboard that shows their pending requests, total completed jobs, and current earnings at a glance. A prominent availability toggle — which saves across sessions — lets workers quickly switch between accepting and pausing new work.

JOB MANAGEMENT

Incoming booking requests appear as cards. Workers can accept or reject each one individually. Once a job is underway, they can mark it as complete, which updates the customer's booking status in real time. A full work history page and a matching notification system round out the worker side.

2. Component Architecture

Data Flow

The application follows a clean layered architecture. Data originates from the DummyJSON API, flows through a dedicated service layer that handles HTTP requests and response mapping, is stored in Pinia stores for global state, and is then consumed by Vue components for rendering.

DummyJSON API → Service Layer → Pinia Stores → Vue Components

Layout Structure

App.vue wraps a PageWrapper component that acts as the shell for all pages. The shell contains a fixed TopBar (with an avatar and notification bell), a SideNav that appears only on desktop, and a BottomNav for mobile users. The active page renders inside a RouterView slot in the centre.

Customer Pages

- **HomeView.vue** — advertisement carousel, platform stats, service category grid, top worker cards, how-it-works section, testimonials
- **ServicesView.vue** — debounced search, category filter bar, service cards grid, worker list, and the BookingModal with its three steps
- **WorkerDetailView.vue** — avatar, star rating, badges, recent jobs list, booking bottom sheet

Worker Pages

- **WorkerHomeView.vue** — availability toggle, stats cards (pending / completed / earnings), pending booking cards with accept & reject actions

State Management — Pinia Stores

- **auth.store** — login, logout, session persistence, token, and role
- **booking.store** — create bookings, update status, full booking history saved to localStorage
- **service.store** — worker and service data, search, and category filtering
- **notification.store** — alerts and unread count management
- **ui.store** — dark mode toggle and toast messages

TypeScript Interfaces

All data shapes are defined in strongly-typed interfaces under `/src/types/`. This makes the codebase self-documenting and catches mismatches at compile time rather than runtime.

- **api.types.ts** — raw DummyJSON API response shapes
- **user.types.ts** — BaseUser, Worker, Customer, UserRole
- **booking.types.ts** — Booking, BookingStatus, CreateBookingPayload
- **service.types.ts** — Service, ServiceFilter
- **notification.types.ts** — AppNotification, NotificationType
- **auth.types.ts** — LoginPayload, StoredSession

Reusable Common Components

A library of eight shared components keeps the UI consistent throughout the application without duplicating code: AppButton, AppInput, AppModal, AppToast, AppAvatar, AppBadge, AppSkeleton, and AppRating. Each is generic enough to be dropped into any view with minimal configuration.

3. Bonus Features

Authentication Simulation

Rather than mocking login with hardcoded credentials, the app makes a real POST request to DummyJSON's `/auth/login` endpoint. The JWT token returned is treated exactly as it would be in a production app —

stored securely in localStorage and attached to subsequent API calls. This gave the project a realistic auth flow without the overhead of a custom backend.

Pinia Global State with Persistence

The booking store serialises all booking data to localStorage after every mutation. This means a customer's booking history — and a worker's accepted jobs — are still there after closing the tab and coming back. It's a small detail that makes the app feel genuinely usable rather than just a demo.

Dynamic Routing with Navigation Guards

The /worker-profile/:id route is fully dynamic, pulling the worker ID from the URL and fetching that individual's data. Vue Router navigation guards check the user's role before allowing access to any page, automatically redirecting customers away from worker-only routes and vice versa.

Dark Mode

Dark mode is implemented with Tailwind's class strategy and the dark: variant, meaning the entire colour system flips at the root level without duplicating any component styles. The user's preference is saved to localStorage so it carries over to future sessions.

4. API Endpoints Used

POST /auth/login

Handles user login. The response includes a JWT access token along with the user's basic profile, which gets mapped into the internal StoredSession interface. The token is saved to localStorage and used to persist the session on page reload.

GET /users?limit=30

Fetches the pool of worker profiles. Since DummyJSON users aren't categorised as tradespeople, deterministic computed fields are derived from each user's data — things like hourly rate and service category — so the workers list feels realistic and varied without manual seeding.

GET /users/:id

Powers the worker detail page. When a customer taps on a specific professional, their full profile is fetched by ID, giving access to bio, contact details, and any additional fields used to build the rich profile view.